



Evolutionary Computation Assignment 2: Greedy Regret Heuristics

BENGRICHE Tidiane (ER-2037)
SZCZUBLIŃSKA Joanna Wiktorja (156070)



Contents

1	Problem	1
2	Pseudocode	1
3	Results	4
4	Conclusion	12

1 Problem

The goal of this week is to improve the Greedy Cycle and NNFlexible algorithms from last week by adding two features:

- Greedy 2-regret heuristics.
- Greedy heuristics with a weighted sum criterion – 2-regret + best change of the objective function.

The purpose of this report is therefore to present to you the algorithms implemented in C++ and to see how these new features improve the results of last week.

You can see the repository of our assignment on github:

https://github.com/tbengric/EC_laboratory

2 Pseudocode

In this section, we will present the pseudocodes of the algorithms. For this assignment, we will code in C++.

Nearest Neighbor K-Regret Weighted Heuristic (Part 1)

```
FUNCTION nearestNeighborKRegretWeighted(dist, nodes, startNodeId,
kRegret = 2, weightRegret = 0.5):
  path ← [startNodeId]
  maxSize ← FLOOR(LENGTH(nodes) / 2)
  visited ← array of FALSE with size LENGTH(nodes)
  visited[startNodeId] ← TRUE
  weightObjective ← 1.0 - weightRegret
  WHILE LENGTH(path) < maxSize DO
    bestNode ← -1
    bestPos ← -1
    bestScore ← -INF
    FOR each node IN nodes DO
      IF visited[node.id] THEN CONTINUE
      insertionCosts ← []
      FOR i FROM -1 TO LENGTH(path)-1 DO
        IF i = -1 THEN
          cost ← dist[node.id][path[0]] + node.cost
        ELSE IF i = LENGTH(path)-1 THEN
          cost ← dist[path[-1]][node.id] + node.cost
        ELSE
          cost ← dist[path[i]][node.id] + dist[node.id][path[i+1]] -
dist[path[i]][path[i+1]] + node.cost
        END IF
        APPEND (cost, i+1) TO insertionCosts
      END FOR
    END FOR
```

Nearest Neighbor K-Regret Weighted Heuristic (Part 2)

```
    SORT insertionCosts BY cost ASCENDING
    regret  $\leftarrow$  SUM(insertionCosts[j].cost - insertionCosts[0].cost FOR j = 1 TO
kRegret-1)
    weightedScore  $\leftarrow$  weightRegret * regret - weightObjective *
insertionCosts[0].cost
    IF weightedScore > bestScore THEN
        bestScore  $\leftarrow$  weightedScore
        bestNode  $\leftarrow$  node.id
        bestPos  $\leftarrow$  insertionCosts[0].position
    END IF
END FOR
INSERT bestNode AT bestPos IN path
visited[bestNode]  $\leftarrow$  TRUE
END WHILE
APPEND path[0] TO path
RETURN path
```

Greedy Cycle K-Regret Weighted (Part 1)

```
FUNCTION greedyCycleKRegretWeighted(dist, nodes, startNodeId,
kRegret = 2, weightRegret = 0.5)
    path  $\leftarrow$  [startNodeId]
    numToSelect  $\leftarrow$  FLOOR(LENGTH(nodes) / 2)
    visited  $\leftarrow$  array of FALSE with size LENGTH(nodes)
    visited[startNodeId]  $\leftarrow$  TRUE
    weightObjective  $\leftarrow$  1.0 - weightRegret
    bestSecondNode  $\leftarrow$  -1
    bestInitialScore  $\leftarrow$  INF
    FOR each node IN nodes DO
        IF visited[node.id] THEN CONTINUE
        score  $\leftarrow$  2 * dist[startNodeId][node.id] + nodes[startNodeId].cost + node.cost
        IF score < bestInitialScore THEN
            bestInitialScore  $\leftarrow$  score
            bestSecondNode  $\leftarrow$  node.id
        END IF
    END FOR
    APPEND bestSecondNode TO path
    visited[bestSecondNode]  $\leftarrow$  TRUE
    APPEND startNodeId TO path
```

Greedy Cycle K-Regret Weighted (Part 2)

```
WHILE LENGTH(path) < numToSelect + 1 DO
  bestNode ← -1
  bestPos ← -1
  bestScore ← -INF
  FOR each node IN nodes DO
    IF visited[node.id] THEN CONTINUE
    insertionCosts ← []
    FOR i FROM 0 TO LENGTH(path)-2 DO
      u ← path[i]
      v ← path[i+1]
      cost ← dist[u][node.id] + dist[node.id][v] - dist[u][v] + node.cost
      APPEND (cost, i+1) TO insertionCosts
    END FOR
    SORT insertionCosts BY cost ASCENDING
    regret ← SUM(insertionCosts[j].cost - insertionCosts[0].cost FOR j = 1 TO
kRegret-1)
    weightedScore ← weightRegret * regret - weightObjective *
insertionCosts[0].cost
    IF weightedScore > bestScore THEN
      bestScore ← weightedScore
      bestNode ← node.id
      bestPos ← insertionCosts[0].position
    END IF
  END FOR
  INSERT bestNode AT bestPos IN path
  visited[bestNode] ← TRUE
END WHILE
RETURN path
```

3 Results

After trying these different algorithms, we will check their effectiveness by showing the constructed cycle.

In the plot, the cost of each node is represented using a color scale: yellow indicates higher cost, while purple indicates lower cost. Additionally, the edges show the sequence of nodes in the path starting from 0. Below the figure, we displayed the IDs of the nodes in the best path.

Firstly, we will show the previous figures for the previous algorithms :

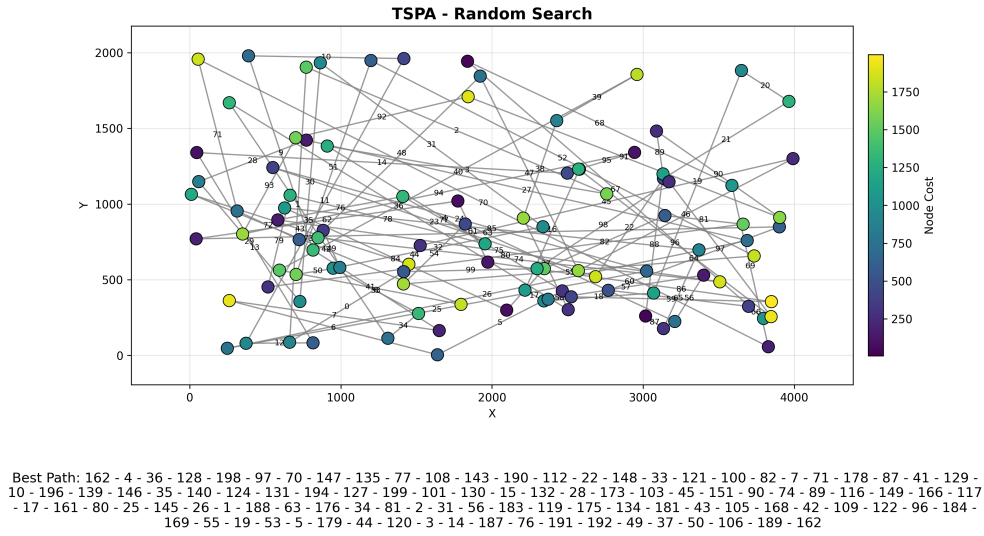


Figure 1: Random Search TSPA

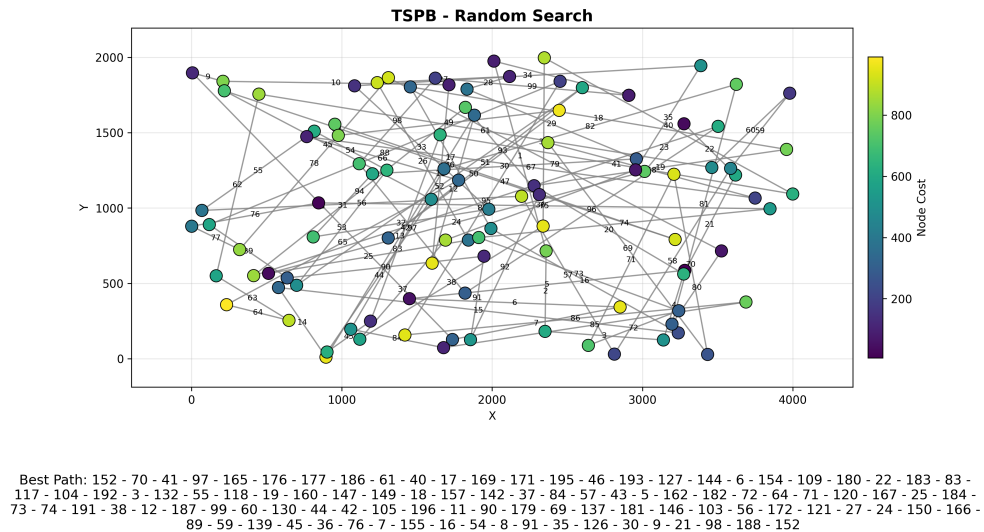


Figure 2: Random Search TSPB

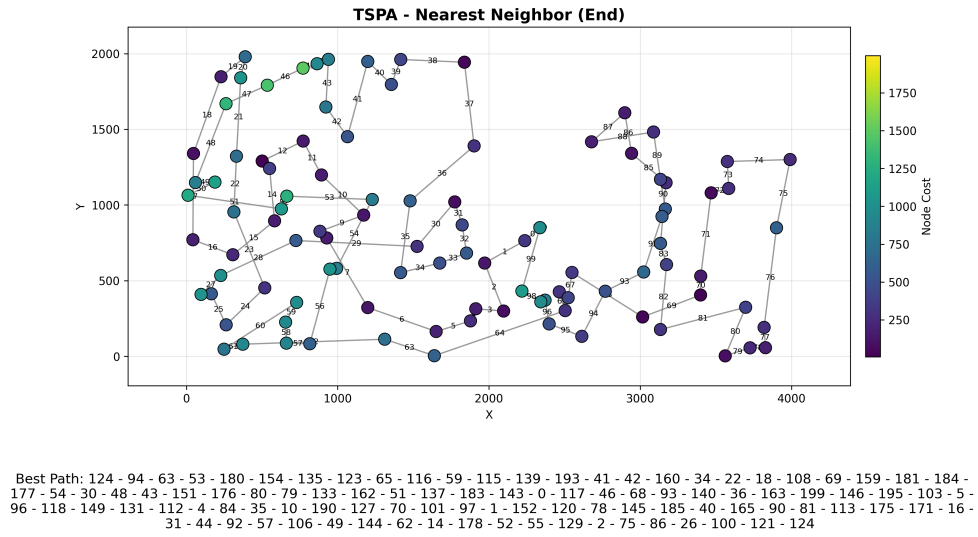


Figure 3: Nearest Neighbor TSPA (End)

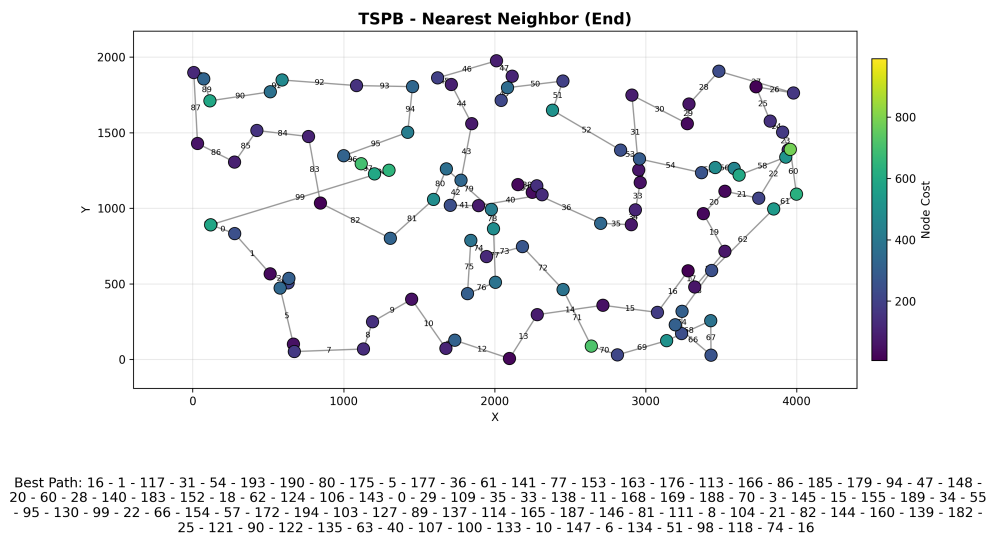
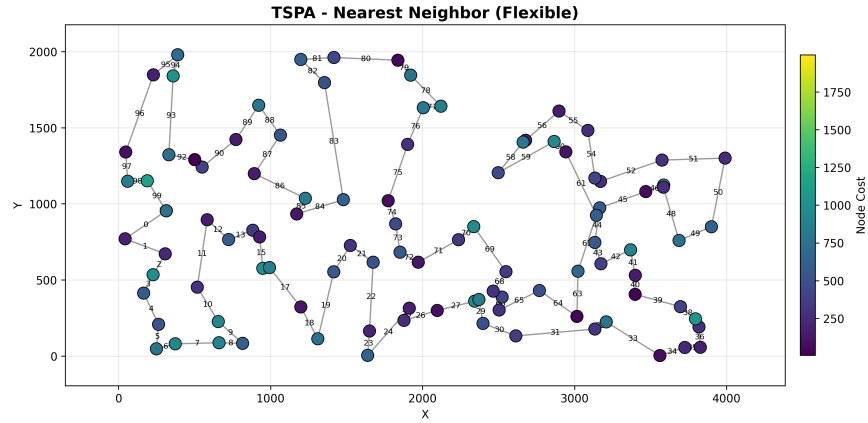
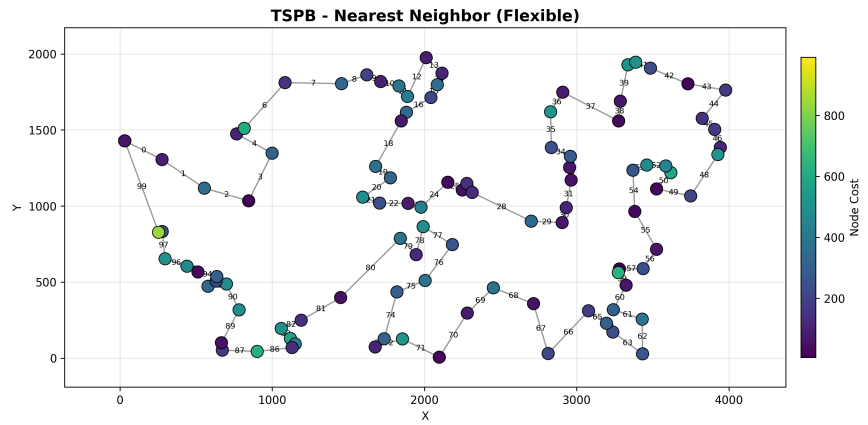


Figure 4: Nearest Neighbor TSPB (End)



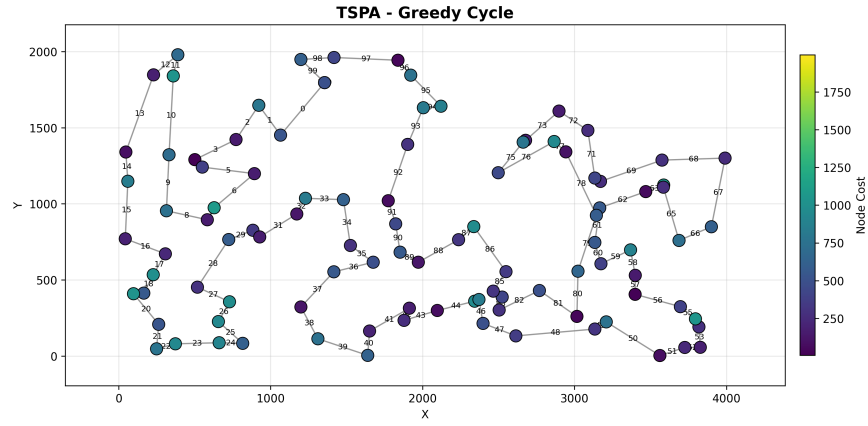
Best Path: 181 - 34 - 160 - 48 - 54 - 177 - 10 - 190 - 4 - 112 - 84 - 184 - 42 - 43 - 116 - 65 - 131 - 149 - 123 - 127 - 162 - 151 - 133 - 135 - 70 - 180 - 154 - 53 - 100 - 26 - 86 - 75 - 44 - 25 - 16 - 171 - 175 - 113 - 56 - 31 - 78 - 145 - 179 - 92 - 57 - 52 - 185 - 119 - 40 - 196 - 81 - 90 - 165 - 106 - 178 - 14 - 144 - 62 - 9 - 148 - 102 - 49 - 55 - 129 - 120 - 2 - 101 - 1 - 97 - 152 - 124 - 94 - 63 - 79 - 80 - 176 - 137 - 23 - 186 - 89 - 183 - 143 - 117 - 0 - 51 - 59 - 118 - 115 - 46 - 68 - 139 - 41 - 193 - 159 - 69 - 108 - 18 - 22 - 146 - 195 - 181

Figure 5: Nearest Neighbor Flexible TSPA



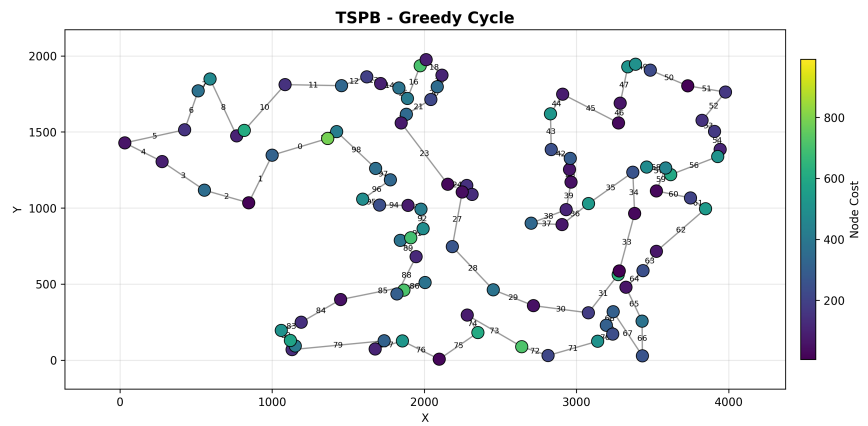
Best Path: 63 - 135 - 131 - 121 - 51 - 90 - 191 - 147 - 6 - 188 - 169 - 132 - 13 - 70 - 3 - 15 - 145 - 195 - 168 - 139 - 11 - 182 - 138 - 33 - 160 - 29 - 0 - 109 - 35 - 143 - 106 - 124 - 62 - 18 - 55 - 34 - 170 - 152 - 183 - 140 - 4 - 149 - 28 - 20 - 60 - 148 - 47 - 94 - 66 - 179 - 185 - 22 - 99 - 130 - 95 - 86 - 166 - 194 - 176 - 180 - 113 - 103 - 114 - 137 - 127 - 89 - 163 - 187 - 153 - 81 - 77 - 141 - 91 - 36 - 61 - 21 - 82 - 111 - 144 - 8 - 104 - 177 - 5 - 45 - 142 - 78 - 175 - 162 - 80 - 190 - 136 - 73 - 193 - 31 - 54 - 117 - 198 - 156 - 1 - 197 - 63

Figure 6: Nearest Neighbor Flexible TSPB



Best Path: 0 - 46 - 68 - 139 - 193 - 41 - 115 - 5 - 42 - 181 - 159 - 69 - 108 - 18 - 22 - 146 - 34 - 160 - 48 - 54 - 30 - 177 - 10 - 190 - 4 - 112 - 84 - 35 - 184 - 43 - 116 - 65 - 59 - 118 - 51 - 151 - 133 - 162 - 123 - 127 - 70 - 135 - 180 - 154 - 53 - 100 - 26 - 86 - 75 - 44 - 25 - 16 - 171 - 175 - 113 - 56 - 31 - 78 - 145 - 179 - 92 - 57 - 52 - 185 - 119 - 40 - 196 - 81 - 90 - 165 - 106 - 178 - 14 - 144 - 62 - 9 - 148 - 102 - 49 - 55 - 129 - 120 - 2 - 101 - 1 - 97 - 152 - 124 - 94 - 63 - 79 - 80 - 176 - 137 - 23 - 186 - 89 - 183 - 143 - 117 - 0

Figure 7: Greedy Cycle TSPA



Best Path: 85 - 51 - 121 - 131 - 135 - 63 - 122 - 133 - 10 - 90 - 191 - 147 - 6 - 188 - 169 - 132 - 13 - 161 - 70 - 3 - 15 - 145 - 195 - 168 - 29 - 109 - 35 - 0 - 111 - 81 - 153 - 163 - 180 - 176 - 86 - 95 - 128 - 106 - 143 - 124 - 62 - 18 - 55 - 34 - 170 - 152 - 183 - 140 - 4 - 149 - 28 - 20 - 60 - 148 - 47 - 94 - 66 - 22 - 130 - 99 - 185 - 179 - 172 - 168 - 194 - 113 - 114 - 137 - 103 - 89 - 127 - 165 - 187 - 146 - 77 - 97 - 141 - 91 - 36 - 61 - 175 - 78 - 142 - 45 - 5 - 177 - 82 - 87 - 21 - 8 - 104 - 56 - 144 - 160 - 33 - 138 - 182 - 11 - 139 - 134 - 85

Figure 8: Greedy Cycle TSPB

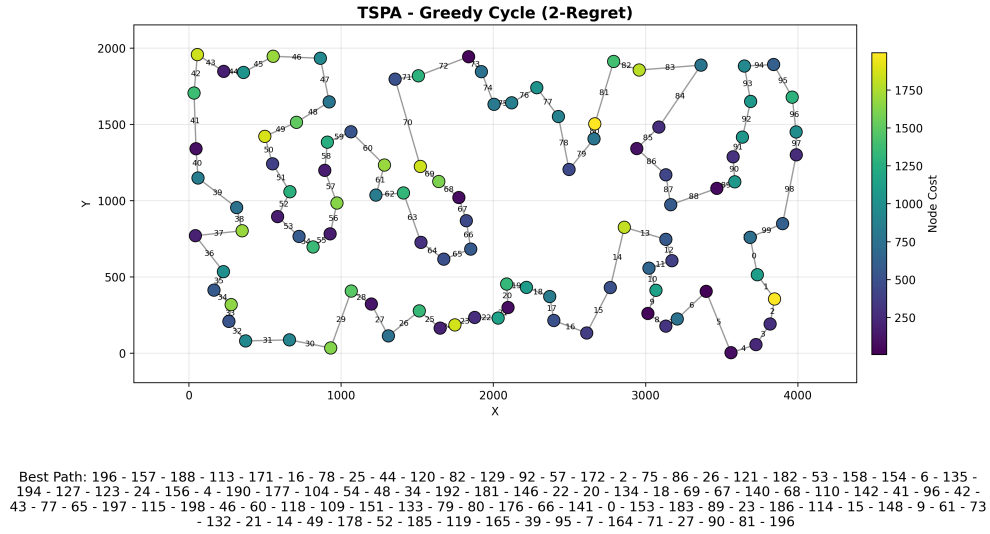


Figure 9: Greedy Cycle 2-Regret TSPA

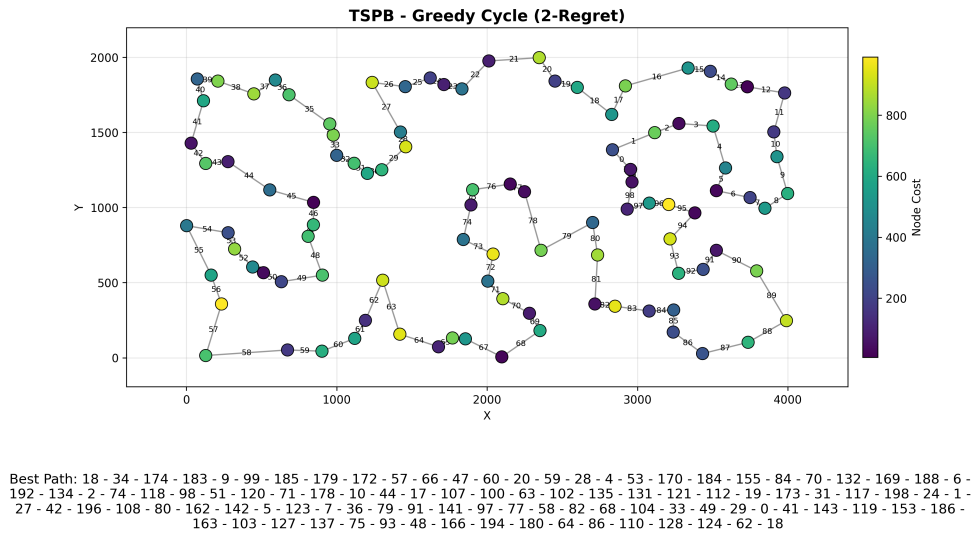


Figure 10: Greedy Cycle 2-Regret TSPB

With the regret feature, we can see that there are no crossing lines and the path seems natural but, as we will see later, the result is higher than without the regret

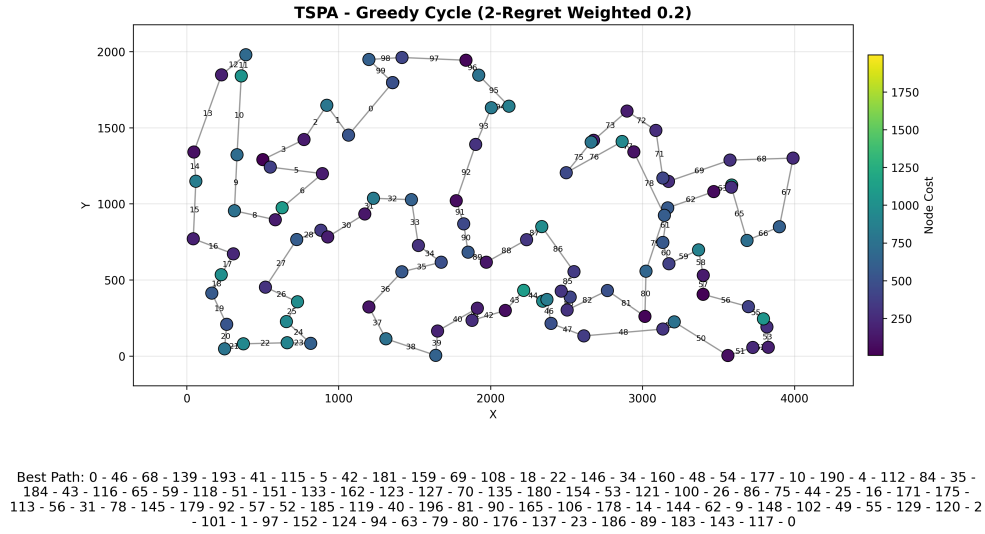


Figure 11: Greedy Cycle 2-Regret Weighted 0.2 TSPA

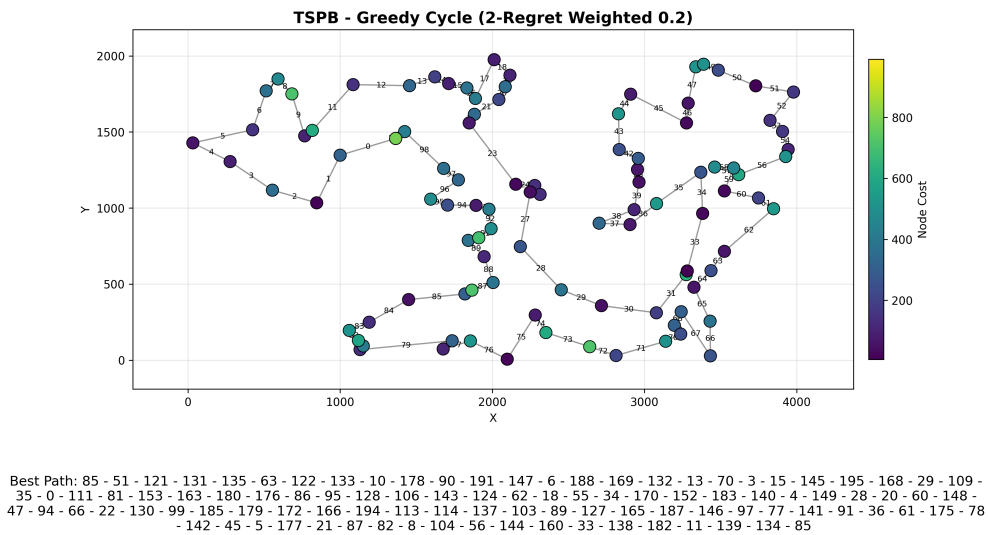


Figure 12: Greedy Cycle 2-Regret Weighted 0.2 TSPB

With both features, the points present are less expensive and thus allow to verify the effectiveness of these two features

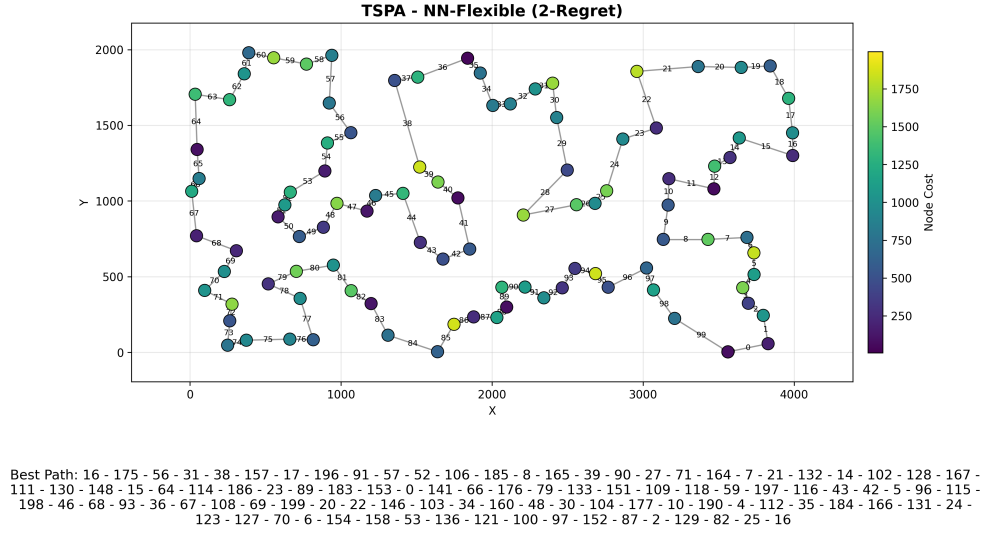


Figure 13: Nearest Neighbor Flexible 2-Regret TSPA

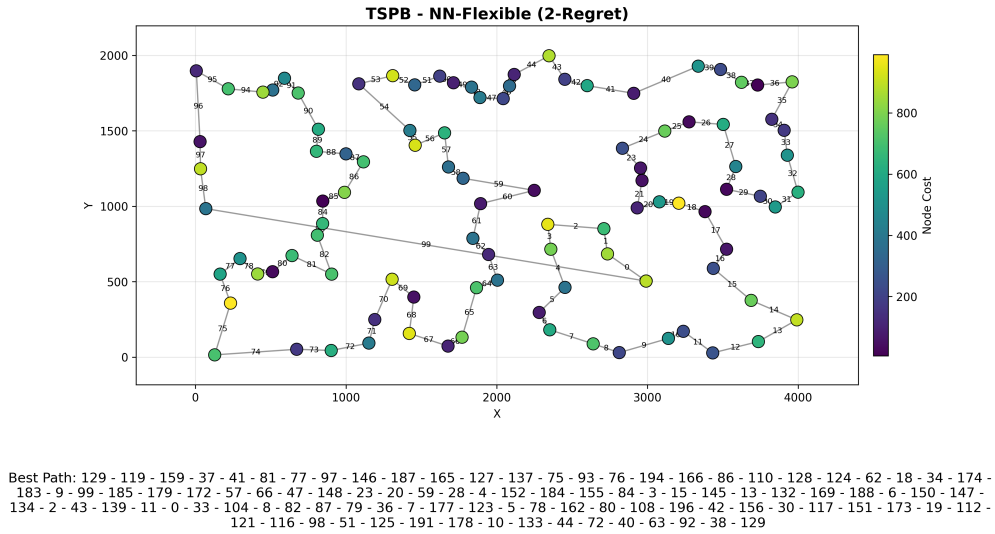


Figure 14: Nearest Neighbor Flexible 2-Regret TSPB

Same as for the greedy Cycle, the paths do not cross but the nodes present are costly

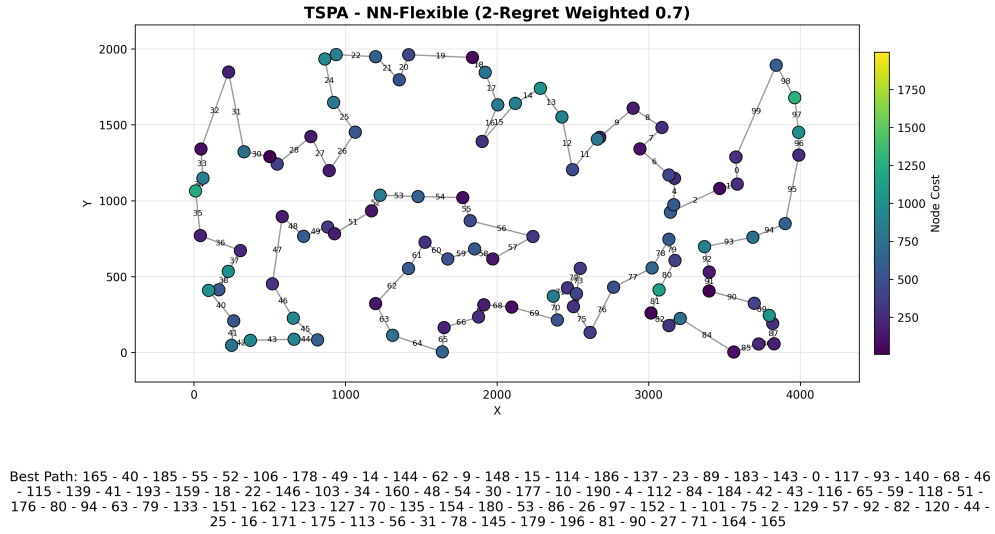


Figure 15: Nearest Neighbor Flexible 2-Regret Weighted 0.7 TSPA

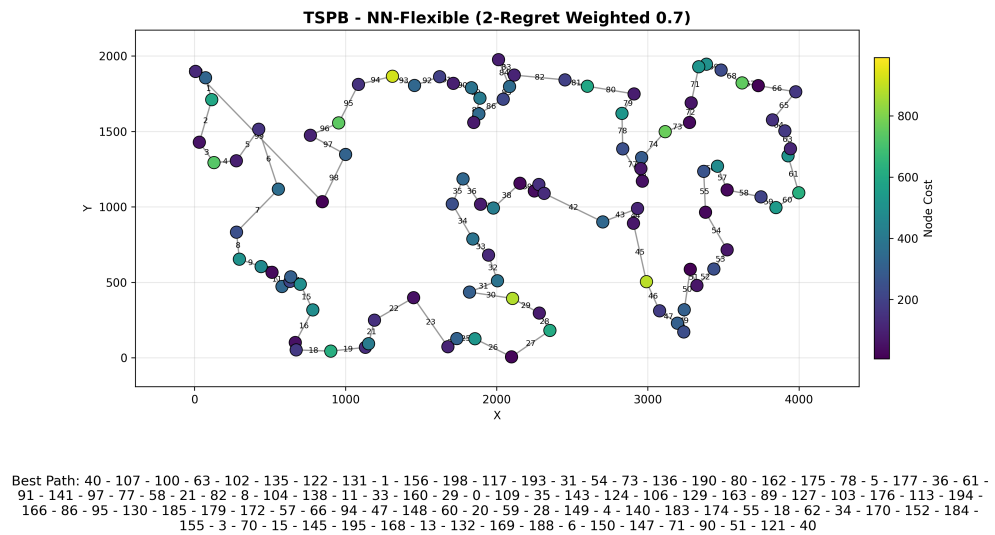


Figure 16: Nearest Neighbor Flexible 2-Regret Weighted 0.7 TSPB

Method	Avg (Min, Max)
Random Search	264654.36 (230472, 296381)
Nearest Neighbor (End)	85108.51 (83182, 89433)
Nearest Neighbor (Flexible)	73594.10 (71868, 75953)
Greedy Cycle	72635.98 (71488, 74410)
Greedy Cycle (2-Regret)	115400.24 (105692, 126860)
Greedy Cycle (3-Regret)	115141.44 (107639, 125196)
Greedy Cycle (2-Regret Weighted 0.5)	72136.15 (71108, 73395)
Greedy Cycle (2-Regret Weighted 0.2)	72452.70 (71525, 73889)
Greedy Cycle (2-Regret Weighted 0.7)	73070.15 (70475, 75365)
NN-Flexible (2-Regret)	117138.49 (108151, 124921)
NN-Flexible (3-Regret)	116369.82 (106776, 124533)
NN-Flexible (2-Regret Weighted 0.5)	72407.59 (70010, 75452)
NN-Flexible (2-Regret Weighted 0.2)	72714.08 (71029, 75182)
NN-Flexible (2-Regret Weighted 0.7)	73582.63 (70566, 77704)

Table 1: Average, minimum, and maximum objective values for TSPA

Method	Avg (Min, Max)
Random Search	213697.41 (187936, 234924)
Nearest Neighbor (End)	54390.43 (52319, 59030)
Nearest Neighbor (Flexible)	48694.25 (44609, 57283)
Greedy Cycle	51400.60 (49001, 57324)
Greedy Cycle (2-Regret)	72712.85 (67809, 78406)
Greedy Cycle (3-Regret)	71738.62 (66912, 75621)
Greedy Cycle (2-Regret Weighted 0.5)	50934.60 (47144, 55700)
Greedy Cycle (2-Regret Weighted 0.2)	50247.52 (48643, 53151)
Greedy Cycle (2-Regret Weighted 0.7)	52424.15 (49944, 56179)
NN-Flexible (2-Regret)	74444.46 (69933, 80278)
NN-Flexible (3-Regret)	72703.85 (66564, 77848)
NN-Flexible (2-Regret Weighted 0.5)	47664.46 (44891, 55247)
NN-Flexible (2-Regret Weighted 0.2)	47434.98 (45145, 52423)
NN-Flexible (2-Regret Weighted 0.7)	50637.08 (47287, 57307)

Table 2: Average, minimum, and maximum objective values for TSPB

Thanks to these tables, we can see that adding Regret and Weighted Sum helps to get better results on Greedy Cycle and NN-Flexible. Unfortunately, only adding the Regret Feature doesn't seem to improve the results.

4 Conclusion

In this study, we compared the previous algorithms for constructing TSP cycles (Random Search, Nearest Neighbor, Nearest Neighbor Flexible, and Greedy Cycle) with the addition of two new features : Regret and Weighted Sum.

By adding only the Regret feature, we realize that the results are not as good as before. However, by adding the Weighted Sum, we realize that we get much better results even

with several different weight values. However, the addition of these features makes the execution time a little longer than previously.